ISTANBUL TOPKAPI ÜNİVERSİTESİ

MÜHENDİSLİK FAKÜLTESİ

YAZILIM MÜHENDİSLİĞİ

Ders Adı:Sunucu Taraflı Web Programlama (FET342)

Ders Başlığı: Maternal Health Risk System

Akademik dönem ve yarıyıl : Bahar 2026

Grup üyeleri:

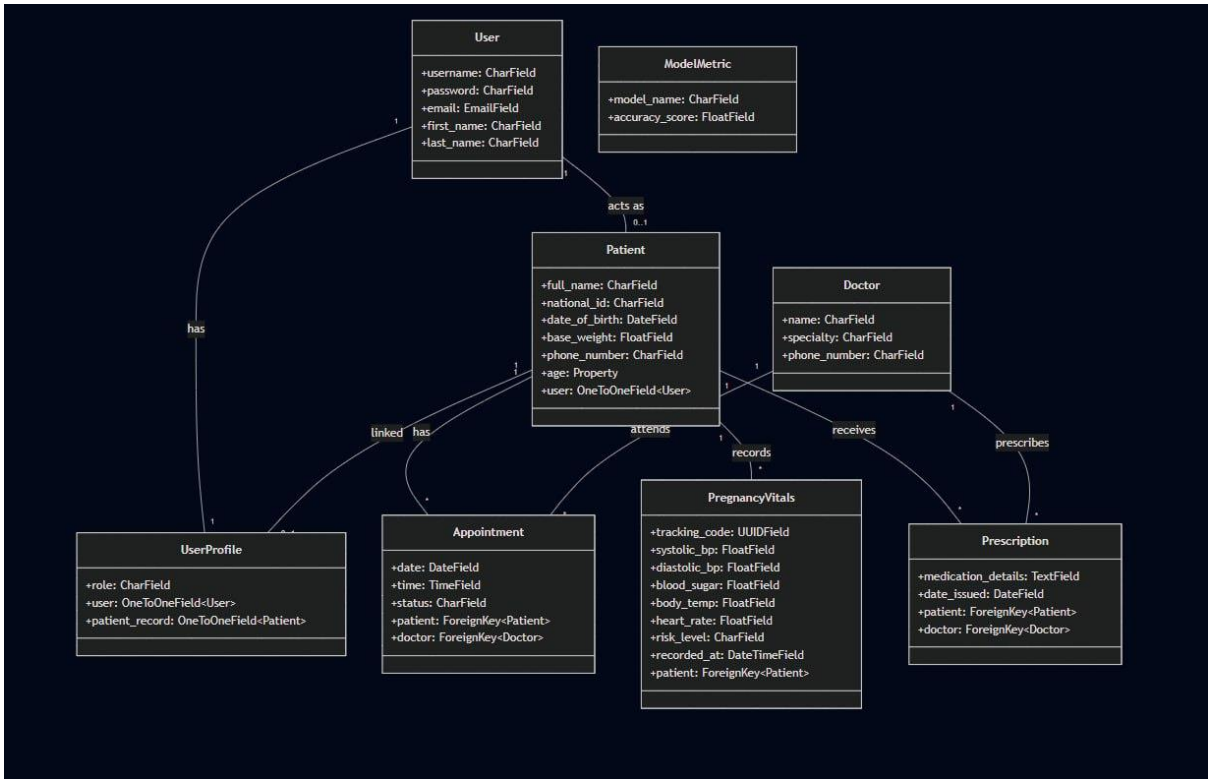
Farih Ömer : 23040301027

Mohamad Ghannam : 22040301194

Obada Alghourani : 23040301013

A. Modellerin Sınıf Diyagramı (Class Diagram)

Projede yer alan veritabanı modellerinin ve aralarındaki ilişkilerin (OneToOne, ForeignKey) kavramsal sınıf diyagramı aşağıda görsel ve kod olarak sunulmuştur:



Mermaid Kod Yapısı:

```
classDiagram
    %% Auth User Model
    class User {
        +username: CharField
        +password: CharField
        +email: EmailField
        +first_name: CharField
        +last_name: CharField
    }

    %% User Profile
    class UserProfile {
        +role: CharField
        +user: OneToOneField~User~
        +patient_record: OneToOneField~Patient~
    }

    %% Core Models
    class Patient {
        +full_name: CharField
        +national_id: CharField
        +date_of_birth: DateField
        +base_weight: FloatField
        +phone_number: CharField
        +age: Property
        +user: OneToOneField~User~
    }

```

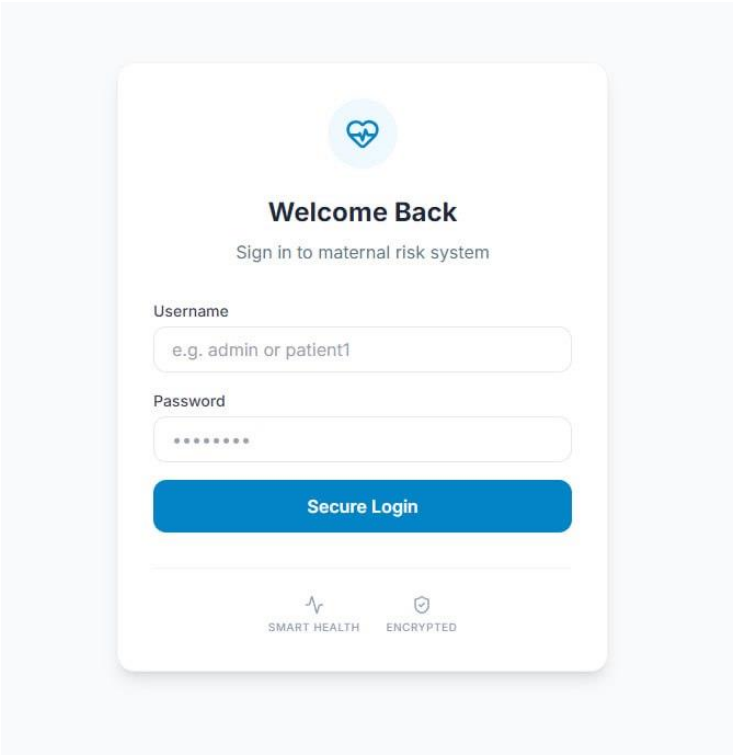
```

    class Doctor {
        +name: CharField
        +specialty: CharField
        +phone_number: CharField
    }

    class Appointment {
        +date: DateField
        +time: TimeField
        +status: CharField
        +patient: ForeignKey~Patient~
        +doctor: ForeignKey~Doctor~
    }

    class PregnancyVitals {
        +tracking_code: UUIDField
        +systolic_bp: FloatField
        +diastolic_bp: FloatField
        +blood_sugar: FloatField
        +body_temp: FloatField
        +heart_rate: FloatField
        +risk_level: CharField
        +recorded_at: DateTimeField
        +patient: ForeignKey~Patient~
    }

    class Prescription {
        +medication_details: TextField
        +date_issued: DateField
        +patient: ForeignKey~Patient~
        +doctor: ForeignKey~Doctor~
    }

    class ModelMetric {
        +model_name: CharField
        +accuracy_score: FloatField
    }

    %% Relationships
    User "1" -- "1" UserProfile : has
    User "1" -- "0..1" Patient : acts as
    Patient "1" -- "0..1" UserProfile : linked
    Patient "1" -- "*" Appointment : has
    Doctor "1" -- "*" Appointment : attends
    Patient "1" -- "*" PregnancyVitals : records
    Patient "1" -- "*" Prescription : receives
    Doctor "1" -- "*" Prescription : prescribes
```

B. UI Sayfalarının Ekran Görüntüleri

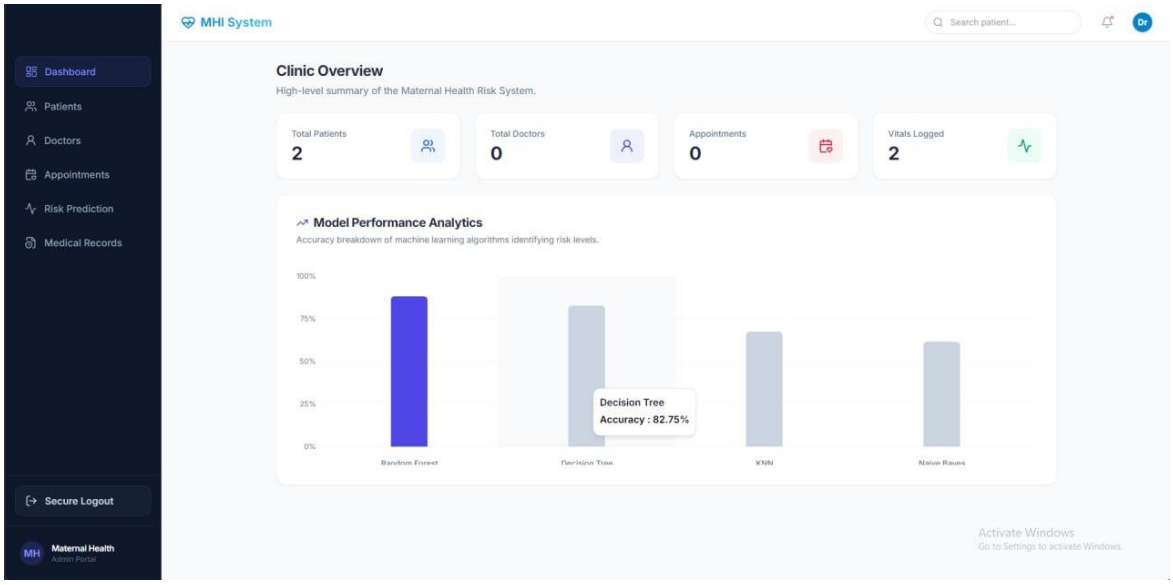
Sistemin Front-end (React) tarafında geliştirilen, kullanıcı rollerine (Admin/Hasta) göre ayrıştırılmış arayüz sayfaları aşağıda listelenmiştir.

1. Kullanıcı Giriş Sayfası (Login)

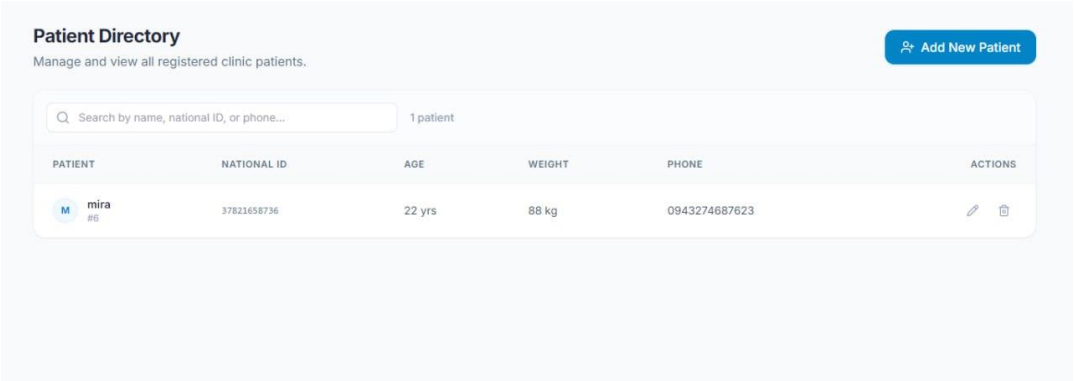
Sisteme Rol tabanlı (Admin veya Patient) güvenli giriş yapılan ekran.

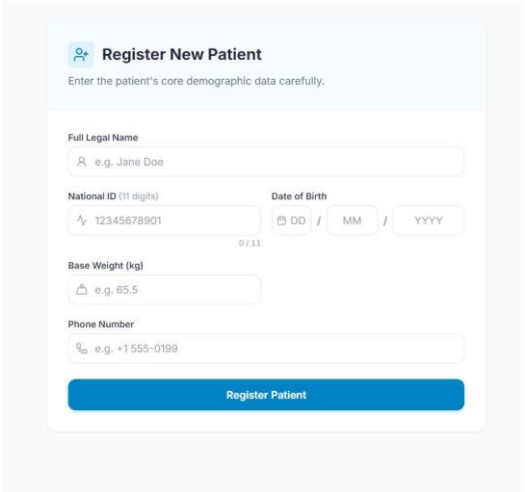


2. Yönetim Paneli (Admin Dashboard) Klinik istatistiklerinin (Hasta sayısı, ölçümler) ve Makine Öğrenmesi algoritmalarının doğruluk oranlarının (Accuracy) dinamik grafiklerle görüntülendiği genel bakış sayfası.

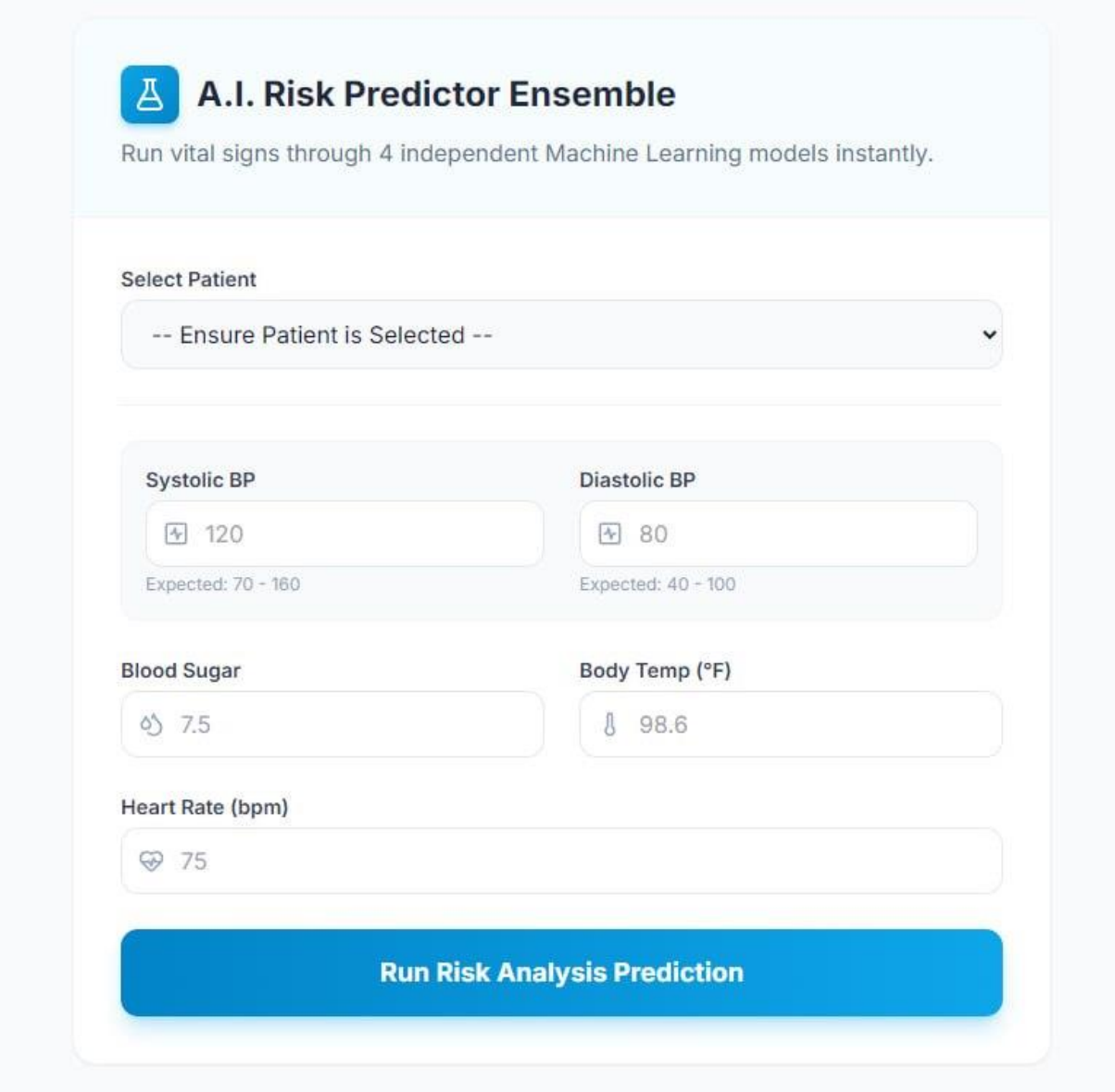


3. Hasta Yönetimi (Patient Directory & Registration) Yeni hastaların demografik ve kimlik bilgileriyle sisteme kaydedildiği ve kayıtlı hastaların filtrelenerek listelendiği yönetim ekranları.

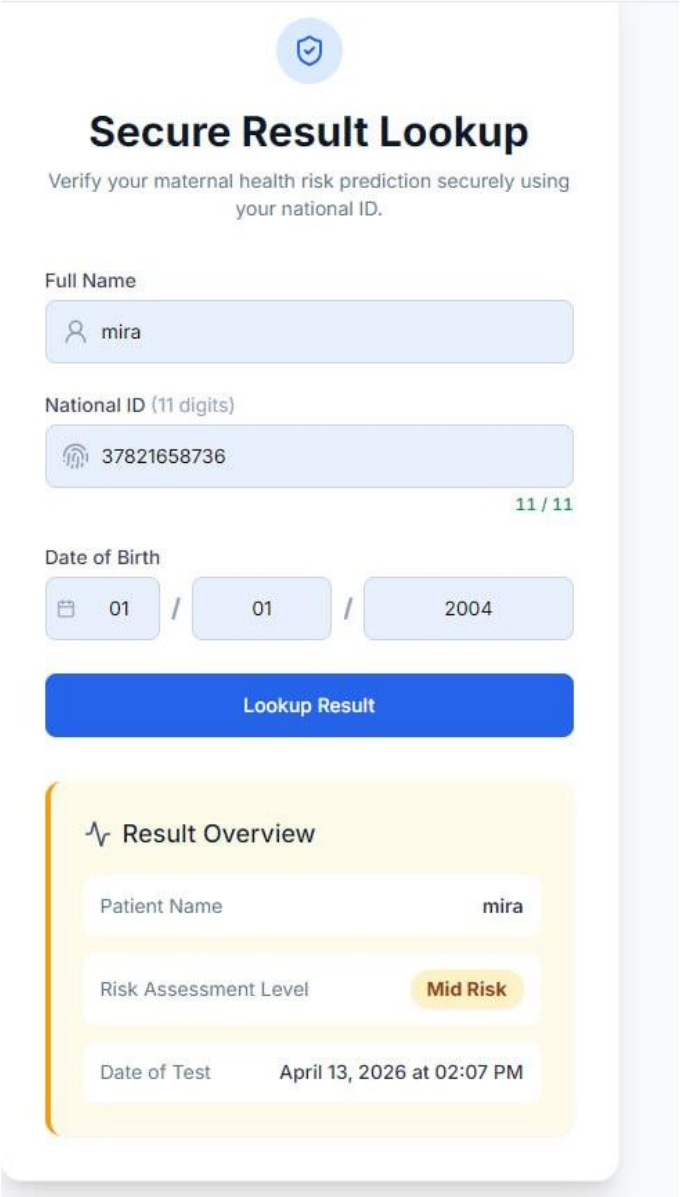




4. Yapay Zeka Risk Analizi (A.I. Predictor & Records) Hastanın anlık ölçümlerinin (Tansiyon, Şeker, Kalp Atışı vb.) sisteme girilerek 4 farklı Makine Öğrenmesi modeliyle değerlendirildiği ve geçmiş risk tahminlerinin listelendiği alan.



5. Güvenli Sonuç Sorgulama Formu (Secure Result Lookup) Hastaların TC Kimlik Numarası, Doğum Tarihi ve İsim bilgilerini girerek kendi risk seviyelerini dışarıdan güvenle doğruladıkları genel erişim sayfası ve sonuç ekranı.



C. Modellerin ve API'lerin Python Kodları

Sistemin Backend (Django & Django REST Framework) altyapısını oluşturan temel bileşenler aşağıdadır.

1. Veritabanı Modelleri (clinic/models.py)

```python
"""
Models for the clinic application.
"""
import uuid
from datetime import date
from django.db import models
from django.conf import settings

class Doctor(models.Model):
    """
    Model representing a doctor.
    """
    name = models.CharField(max_length=255)
    specialty = models.CharField(max_length=255)
    phone_number = models.CharField(max_length=20)

    def __str__(self):
        return f"Dr. {self.name} - {self.specialty}"

class Patient(models.Model):
    """
    Model representing a patient.
    """
    user = models.OneToOneField(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
        null=True,
        blank=True,
        related_name='patient_model'
    )
    full_name = models.CharField(max_length=255)
    national_id = models.CharField(max_length=11, unique=True, null=True, blank=True)
    date_of_birth = models.DateField(null=True, blank=True)
    base_weight = models.FloatField(help_text="Base weight in kg")

    @property
    def age(self):
        if not self.date_of_birth:
            return 0
        today = date.today()
        return today.year - self.date_of_birth.year - ((today.month, today.day) < (self.date_of_birth.month, self.date_of_birth.day))
    phone_number = models.CharField(max_length=20)

    def __str__(self):
        return str(self.full_name)

class Appointment(models.Model):
    """
    Model representing an appointment.
    """
    STATUS_CHOICES = [
        ('Scheduled', 'Scheduled'),
        ('Completed', 'Completed'),
        ('Cancelled', 'Cancelled'),
    ]

    patient = models.ForeignKey(Patient, on_delete=models.CASCADE, related_name='appointments')
    doctor = models.ForeignKey(Doctor, on_delete=models.CASCADE, related_name='appointments')
    date = models.DateField()
    time = models.TimeField()
    status = models.CharField(max_length=50, choices=STATUS_CHOICES, default='Scheduled')

    def __str__(self):
        return f"{self.patient.full_name} - {self.doctor.name} on {self.date}"
```

```python
class PregnancyVitals(models.Model):
    """
    Model representing pregnancy vitals.
    """
    RISK_LEVEL_CHOICES = [
        ('Low', 'Low'),
        ('Mid', 'Mid'),
        ('High', 'High'),
    ]

    patient = models.ForeignKey(Patient, on_delete=models.CASCADE, related_name='vitals')
    tracking_code = models.UUIDField(default=uuid.uuid4, editable=False, unique=True)
    systolic_bp = models.FloatField(help_text="Systolic Blood Pressure (mmHg)")
    diastolic_bp = models.FloatField(help_text="Diastolic Blood Pressure (mmHg)")
    blood_sugar = models.FloatField(help_text="Blood Sugar Level (mg/dL)")
    body_temp = models.FloatField(help_text="Body Temperature (°C or °F)")
    heart_rate = models.FloatField(help_text="Heart Rate (bpm)")
    risk_level = models.CharField(max_length=10, choices=RISK_LEVEL_CHOICES)

    recorded_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f"Vitals: {self.patient.full_name} | Risk: {self.risk_level}"

class Prescription(models.Model):
    """
    Model representing a prescription.
    """
    patient = models.ForeignKey(Patient, on_delete=models.CASCADE, related_name='prescriptions')
    doctor = models.ForeignKey(Doctor, on_delete=models.CASCADE, related_name='prescriptions')
    medication_details = models.TextField()
    date_issued = models.DateField(auto_now_add=True)

    def __str__(self):
        return f"Rx for {self.patient.full_name} by {self.doctor.name}"

class ModelMetric(models.Model):
    """
    Model representing model metrics.
    """
    model_name = models.CharField(max_length=255)
    accuracy_score = models.FloatField()

    def __str__(self):
        return f"{self.model_name}: {self.accuracy_score}%"

class UserProfile(models.Model):
    """
    Model representing a user profile.
    """
    ROLE_CHOICES = [
        ('Admin', 'Admin'),
        ('Patient', 'Patient'),
    ]
    user = models.OneToOneField(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
        related_name='profile'
    )
    role = models.CharField(max_length=20, choices=ROLE_CHOICES, default='Patient')
    patient_record = models.OneToOneField(
        'Patient',
        on_delete=models.SET_NULL,
        null=True,
        blank=True,
        related_name='user_profile'
    )

    def __str__(self):
        return f"{self.user.username} - {self.role}"
```

2. API Yönlendirmeleri (clinic/urls.py)

```python
from django.urls import path, include
from rest_framework.routers import DefaultRouter
from . import views

router = DefaultRouter()
router.register(r'patients', views.PatientViewSet, basename='patient')
router.register(r'vitals-data', views.PregnancyVitalsViewSet, basename='vitals-data')

urlpatterns = [
    # Router mapping for secure ViewSets
    path('', include(router.urls)),

    # Add specific requested endpoint for My Vitals
    path('my-vitals/', views.my_vitals, name='my_vitals'),

    # Doctor Endpoints
    path('doctors/', views.doctor_list, name='doctor_list'),
    path('doctors/create/', views.create_doctor, name='create_doctor'),

    # Appointment Endpoints
    path('appointments/', views.appointment_list, name='appointment_list'),
    path('appointments/create/', views.create_appointment, name='create_appointment'),

    # Vitals and ML Predict Endpoints
    path('vitals/predict/', views.create_vitals_predict, name='create_vitals_predict'),

    # Public Lookup Endpoint
    path('verify-result/', views.verify_patient_result, name='verify_patient_result'),

    # Dashboard Endpoint
    path('dashboard/', views.dashboard_stats, name='dashboard_stats'),

    # Metrics Endpoint
    path('model-metrics/', views.model_metrics_list, name='model_metrics_list'),

    # Auth Endpoints
    path('auth/login/', views.custom_login, name='login'),
    path('auth/me/', views.me, name='me'),
]
```

3. API Kontrolcüleri (clinic/views.py)

```python
from rest_framework.decorators import api_view, permission_classes
from rest_framework.response import Response
from rest_framework import status, viewsets
from django.shortcuts import get_object_or_404
from django.conf import settings
from rest_framework_simplejwt.tokens import RefreshToken
from django.contrib.auth import authenticate
from rest_framework.permissions import IsAuthenticated, AllowAny
import os
import joblib

from .permissions import IsAdminOrStaff, IsPatient, IsOwner
from .models import Doctor, Patient, Appointment, PregnancyVitals, Prescription, ModelMetric
from .serializers import (
    DoctorSerializer, PatientSerializer, AppointmentSerializer,
    PregnancyVitalsSerializer, VitalsPredictSerializer, ModelMetricSerializer
)

class PatientViewSet(viewsets.ModelViewSet):
    serializer_class = PatientSerializer

    def get_permissions(self):
        if self.action in ['list', 'create']:
            permission_classes = [IsAuthenticated]
        else:
            permission_classes = [IsAuthenticated, IsOwner]
        return [permission() for permission in permission_classes]

    def get_queryset(self):
        user = self.request.user
        if hasattr(user, 'profile') and user.profile.role == 'Admin':
            return Patient.objects.all().order_by('-id')
        return Patient.objects.filter(user=user).order_by('-id')

    def perform_create(self, serializer):
        user = self.request.user
        if hasattr(user, 'profile') and user.profile.role == 'Admin':
            serializer.save()
        else:
            serializer.save(user=user)

class PregnancyVitalsViewSet(viewsets.ModelViewSet):
    serializer_class = PregnancyVitalsSerializer

    def get_permissions(self):
        if self.action in ['list', 'create']:
            permission_classes = [IsAuthenticated]
        else:
            permission_classes = [IsAuthenticated, IsOwner]
        return [permission() for permission in permission_classes]

    def get_queryset(self):
        user = self.request.user
        if hasattr(user, 'profile') and user.profile.role == 'Admin':
            return PregnancyVitals.objects.all().order_by('-recorded_at')
        return PregnancyVitals.objects.filter(patient__user=user).order_by('-recorded_at')


@api_view(['GET'])
@permission_classes([IsAuthenticated])
def my_vitals(request):
    vitals = PregnancyVitals.objects.filter(patient__user=request.user).order_by('-recorded_at')
    serializer = PregnancyVitalsSerializer(vitals, many=True)
    return Response(serializer.data)

@api_view(['POST'])
def custom_login(request):
    username = request.data.get('username')
    password = request.data.get('password')
    user = authenticate(username=username, password=password)
    if user:
        refresh = RefreshToken.for_user(user)
        role = user.profile.role if hasattr(user, 'profile') else 'Visitor'

        patient_record_id = None
        if hasattr(user, 'patient_model') and user.patient_model:
            patient_record_id = user.patient_model.id

        return Response({
            'refresh': str(refresh),
            'access': str(refresh.access_token),
            'role': role,
            'patient_id': patient_record_id,
            'name': user.get_full_name() or user.username
        })
    return Response({'error': 'Invalid Credentials'}, status=status.HTTP_401_UNAUTHORIZED)
```